

Relatório 5 - Prática de React e Next.js

Kaíque Medeiros Lima

1. Descrição da atividade

a. Componentes e organização do gerenciador de tarefas

A primeira parte da atividade é um gerenciador de tarefas em React, feito com Vite a partir do curso indicado no card. A aplicação foi dividida em componentes como AddTask, Tasks e Title, além de componentes reutilizáveis para os campos e botões. O App guarda o estado principal e passa os dados e as funções para os componentes pelas props, então cada parte da interface fica com uma responsabilidade definida.

O AddTask controla os campos de título e descrição com useState. Antes de adicionar, o código confere se os dois campos foram preenchidos, usando trim para não aceitar só espaços, e avisa quando falta algum. Depois da inclusão os campos são limpos. A nova tarefa recebe um identificador com crypto.randomUUID e começa com isCompleted igual a false.

b. Estado, eventos e dados da API

As ações da lista mostram como o estado acompanha a interação. Para marcar uma tarefa como concluída, o código usa map e troca isCompleted apenas no item clicado. Para excluir, usa filter e mantém os itens com identificador diferente. Um useEffect salva a lista no localStorage sempre que ela muda, e o estado inicial lê esses dados com JSON.parse dentro de um try, voltando para uma lista vazia se o conteúdo salvo estiver corrompido.

O botão Carregar tarefas da API busca dez tarefas no JSONPlaceholder com fetch e substitui a lista atual por elas. Enquanto a requisição acontece, o botão fica desabilitado e mostra Carregando tarefas, e se a resposta não for bem-sucedida aparece uma mensagem de erro na tela. A classe de tarefa concluída é montada com um operador ternário, para que uma tarefa pendente não receba a palavra false como classe.



Figura 1 - Lista com tarefas carregadas da API, uma adicionada à mão e duas concluídas. Autoria própria, navegador.

c. Navegação e apresentação dos detalhes

O projeto usa o React Router para separar a página inicial da rota `/task`. Ao abrir os detalhes, o título e a descrição vão para a URL por meio de `URLSearchParams`, e na outra página o `useSearchParams` recupera esses valores para mostrar o conteúdo. O botão de voltar usa a navegação do histórico. A interface usa Tailwind CSS e ícones do lucide-react, e a tarefa concluída recebe um ícone de confirmação e o texto riscado.



Figura 2 - Página de detalhes da tarefa, com os dados vindos da URL. Autoria própria, navegador.

d. Miniblog com Next.js e organização por módulos

A segunda parte é um miniblog de notícias sobre Fórmula 1, feito com Next.js, TypeScript e Tailwind. A pasta `app` organiza as rotas: a home, a rota dinâmica `/artigos/[slug]`, o grupo (`auth`) com login e cadastro e o grupo (`secure`) com o perfil. As telas e os recursos de cada funcionalidade ficam nos módulos `blog`, `auth` e `profile`, e componentes como `PageShell`, `SiteHeader`, `SiteFooter`, `TextField` e `PrimaryButton` ficam em `shared` para serem reaproveitados.

A home e o detalhe são `Server Components`: buscam as notícias no servidor e entregam os dados prontos para as telas. As partes que usam estado e eventos, como a lista com busca, o botão de favoritos, os formulários e o perfil, são marcadas com `use client`. Assim a página continua simples e só os componentes interativos vão para o navegador.

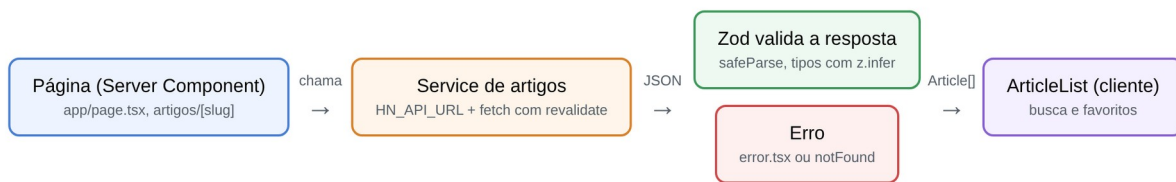


Figura 3 - Caminho dos dados da API até a lista do miniblog. Autoria própria.

e. Consumo da API e validação com Zod

As notícias vêm da API de busca da Hacker News. O endereço base não fica no código: é lido da variável de ambiente `HN_API_URL`, que tem um exemplo em `.env.example`, e o service lança um erro claro se ela não estiver definida. O fetch usa `revalidate` de uma hora, então a mesma busca não é repetida a cada acesso. Como a resposta vem de fora, ela é validada em tempo de execução com um schema do Zod, e o tipo usado no código é derivado dele com `z.infer`, em vez de uma asserção de tipo que o compilador aceitaria sem conferir nada.

O slug da rota dinâmica também é tratado como entrada externa: antes de montar a busca, o código confere se ele é numérico, e um valor como `abc` leva direto à página de não encontrado. Uma notícia inexistente também devolve `null` e chama `notFound`. Já uma falha da API, como um erro 500 ou uma resposta em formato inesperado, lança um erro, que o `error.tsx` mostra com a opção de tentar de novo. Assim uma instabilidade do servidor não aparece para quem usa como se a notícia não existisse.

```

src/modules/blog/data/services/articles/index.ts (trecho)

1  const hnSearchSchema = z.object({
2    hits: z.array(hnHitSchema),
3  });
4
5  type HnHit = z.infer<typeof hnHitSchema>;
6
7  const result = hnSearchSchema.safeParse(await response.json());
8
9  if (!result.success) {
10   throw new Error("A Hacker News respondeu num formato inesperado.");
11 }
12
13 export const fetchArticleBySlug = async (slug: string) => {
14   if (!/^d+$/.test(slug)) return null;
15
16   const [article] = await searchStories({ tags: `story,story_${slug}` });
17
18   return article ?? null;
19 };
  
```

Figura 4 - Validação da resposta e do slug no service de artigos. Código da branch do Card 05.

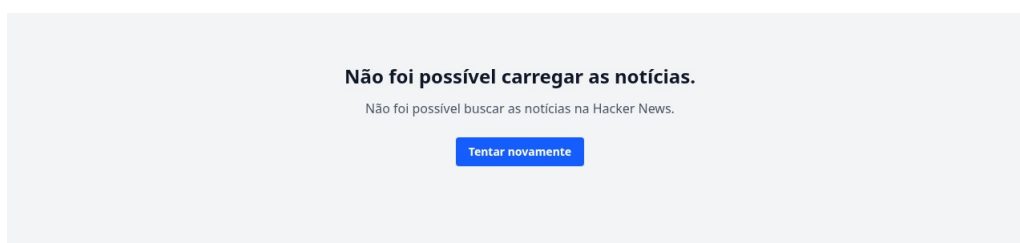


Figura 5 - Tela de erro quando a API não responde corretamente. Autoria própria, navegador.

f. Hooks e interação no miniblog

A lista de artigos usa `useState` para o texto da busca e para o filtro de favoritas, e filtra os títulos sem diferenciar maiúsculas e minúsculas. Quando nada é encontrado, aparece uma mensagem no lugar da lista. O `useRef` guarda a referência do campo de busca, e o botão Focar coloca o cursor nele. Cada card mostra pontos, comentários, data e autor, e usa o mesmo componente `FavoriteButton` da página de detalhe.

Os favoritos ficam no `localStorage` e são lidos pelo hook `useFavorites`, que usa `useSyncExternalStore`. Com isso, todos os botões e a lista enxergam o mesmo valor, a marcação feita no card aparece na página de detalhe e a renderização no servidor não tenta acessar o `localStorage`, que só existe no navegador.

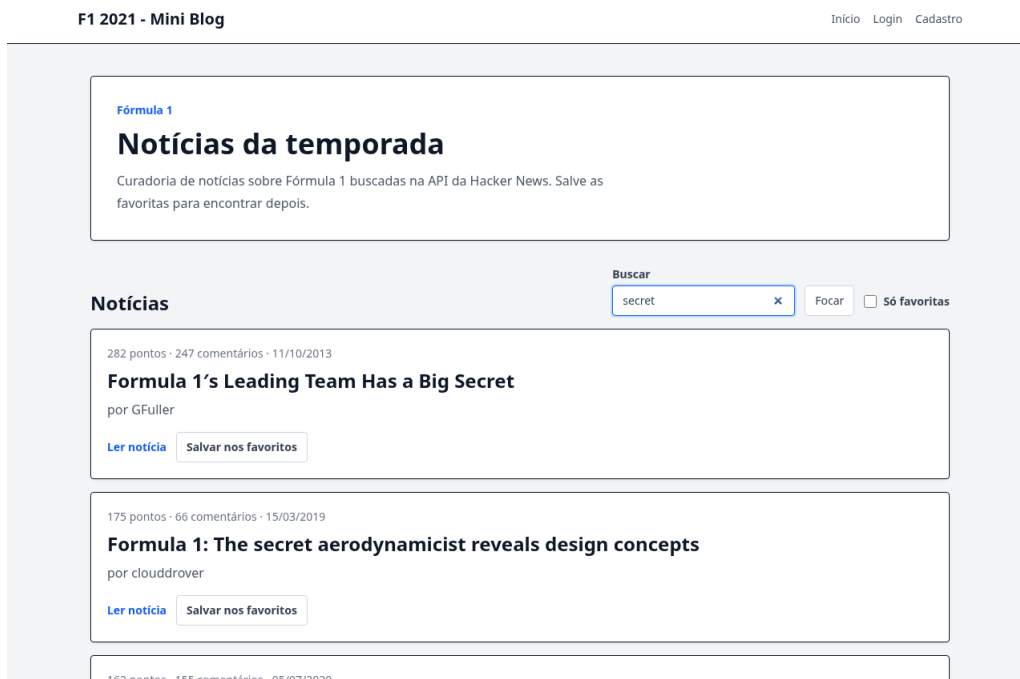


Figura 6 - Busca por título na lista de notícias. Autoria própria, navegador.

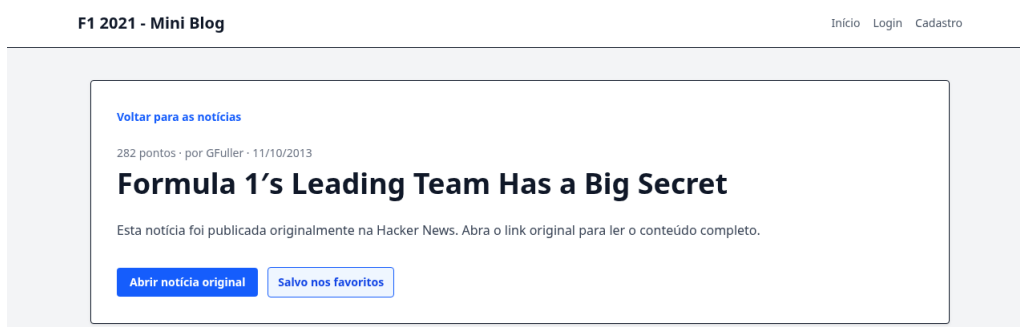


Figura 7 - Página de detalhe de uma notícia salva nos favoritos. Autoria própria, navegador.

g. Login, sessão e área protegida

O login é uma server action que recebe o e-mail e a senha digitados e compara com uma conta de demonstração, documentada no README, porque o projeto não tem banco de usuários. Se os dados não batem, a tela mostra E-mail ou senha incorretos. Se batem, a action grava um cookie de sessão `httpOnly` com validade de uma hora. O layout do grupo

(secure) lê esse cookie no servidor e redireciona para /login quando ele não existe, então o perfil não abre para quem não entrou.

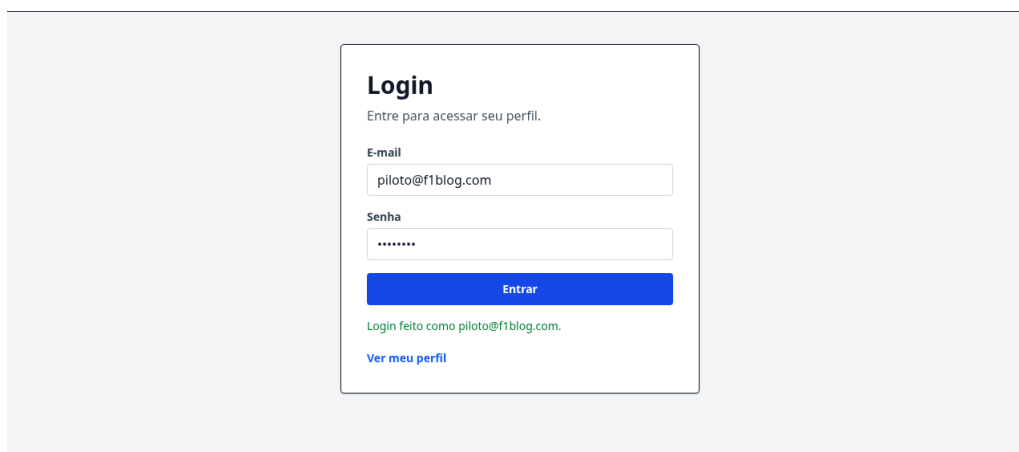
O cabeçalho também lê o cookie e mostra Login e Cadastro para quem está fora, ou Perfil e Sair para quem entrou. O botão Sair chama outra server action, que apaga o cookie e volta para o login. Como o cabeçalho depende do cookie, as páginas passaram a ser renderizadas a cada acesso, e por isso a geração estática com generateStaticParams foi retirada, ficando só o cache do fetch. O cadastro valida o nome e o tamanho mínimo da senha e mostra os dados digitados, mas deixa claro que a conta não é salva. O perfil permite trocar o nome com useState, sem persistência.

```
src/modules/auth/actions.ts (trecho)

1  export const loginAction = async (email: string, password: string) => {
2    if (email !== DEMO_EMAIL || password !== DEMO_PASSWORD) {
3      return false;
4    }
5
6    const cookieStore = await cookies();
7    cookieStore.set({ name: SESSION_COOKIE_NAME, value: "1", httpOnly: true, ... });
8
9    return true;
10 };
11
12 export const logoutAction = async () => {
13   const cookieStore = await cookies();
14
15   cookieStore.delete(SESSION_COOKIE_NAME);
16   redirect("/login");
17 };
```

Figura 8 - Server actions de login e de saída, com as opções do cookie resumidas. Código da branch do Card 05.

Figura 9 - Login recusado com senha errada. Autoria própria, navegador.



Login

Entre para acessar seu perfil.

E-mail

pilotof1blog.com

Senha

Entrar

Login feito como piloto@f1blog.com.

[Ver meu perfil](#)

Figura 10 - Login aceito, com Perfil e Sair no cabeçalho. Autoria própria, navegador.

h. O que é Next.js? Uma Análise do Popular Framework JavaScript

O artigo apresenta o Next.js como um framework baseado em React para construir aplicações web prontas para produção. Além dos componentes do React, ele traz recursos próprios de organização das páginas, roteamento, renderização e otimização. O texto também destaca o suporte a TypeScript e as diferentes formas de renderização, como SSR, geração estática e renderização no cliente.

Um dos pontos principais é o roteamento baseado em arquivos, em que pastas e arquivos viram URLs, incluindo rotas aninhadas e dinâmicas. O artigo também explica a diferença entre gerar uma página antes e renderizar a cada requisição. Na prática, o miniblog mostrou essa escolha: a leitura do cookie no cabeçalho tornou as páginas dinâmicas, enquanto os dados da API continuam reaproveitados pelo cache do fetch.

2. Conclusões

As duas práticas mostram como componentes e hooks trabalham juntos para montar uma interface. No gerenciador, as ações atualizam a lista, o armazenamento local e os dados vindos da API. No Next.js, a separação entre Server e Client Components, o roteamento por pastas e os módulos por domínio deixaram cada parte fácil de encontrar. As correções da revisão reforçaram que dado de fora, seja a resposta da API, o slug da URL ou o que a pessoa digita no formulário, precisa ser validado antes de ser usado, e que um erro do servidor deve aparecer como erro, não como página inexistente.

3. Referências

<https://www.youtube.com/watch?v=e6FigV2fLC8&t=77s>

<https://www.youtube.com/watch?v=2RWsLmu8yVc&t=237s>

<https://kinsta.com/pt/blog/next-js/>

<https://nextjs.org/docs/app/api-reference/functions/cookies>

<https://zod.dev/>